DISCORD BOT — SETUP GUIDE

A live-updating Discord embed for loot resets, daily resets, Starfall cycles, phase changes, vendor resets, and Hal's Moving House — setup, commands, and cloud hosting.

Posts a live-updating embed tracking:

- **Loot Reset** — every 4 hours, from a customizable start time
- **Daily Reset** — once a day, at a customizable time
- **Starfall Cycle** — active for the first 30 minutes of every hour
- **Phase** — a cycle of 4-7 phases, each with its own length in days; stops after the last phase until an admin starts a new cycle
- **Vendor/Commission Reset** — weekly
- **Hal's Moving House** location

The embed is edited once a minute so the countdowns stay current.

Commands and what they do

All five commands require the "Manage Server" permission (regular members can't run them). Type `/` in a channel where the bot is present to see them in Discord's command picker.

Command	Options	Effect
`/ohsetlootreset`	`hour` (0-23), `minute` (0-59)	Changes what clock time the 4-hour loot reset cycle is anchored to. The reset then repeats every 4 hours from that point.
`/ohsetdailyreset`	`hour` (0-23), `minute` (0-59)	Changes what time the once-a-day reset happens.
`/ohsetphase`	`start_date`, `start_time`, `durations`, `names` (optional)	Starts a brand new phase cycle: phase 1 begins at the given date/time, and each phase lasts the number of days you list in `durations` (4 to 7 comma-separated values). Once the last phase ends, the cycle stops until you run this command again.
`/ohcreatetimerpost`	none	Posts a new tracker embed in the channel you run it in, and remembers that channel/message so it can keep updating it every minute. Running it again elsewhere moves the tracker to the new channel.
`/ohsethallocation`	`location`	Updates the text shown for Hal's Moving House location.

EXAMPLE — `/ohsetlootreset`

```
/ohsetlootreset hour:1 minute:30
```

Loot resets now happen at 1:30, 5:30, 9:30, 13:30, 17:30, and 21:30 (every 4 hours from 1:30), instead of the previous anchor time.

EXAMPLE — `/ohsetdailyreset`

```
/ohsetdailyreset hour:9 minute:0
```

The daily reset now happens at 9:00 AM every day (in the `TIMEZONE` set in your config), instead of whatever time it was set to before.

EXAMPLE — `/ohsetphase`

```
/ohsetphase start_date:2026-09-01 start_time:00:00 durations:3,4,3,5 names:Growth,Bloom,Decay,Dormant
```

Starts a 4-phase cycle beginning midnight on Sept 1, 2026: "Growth" runs for 3 days, then "Bloom" for 4 days, then "Decay" for 3 days, then "Dormant" for 5 days (15 days total). After "Dormant" ends, the embed shows "Cycle complete" until `/ohsetphase` is run again. Leaving out `names` would label them "Phase 1", "Phase 2", "Phase 3", "Phase 4" instead.

EXAMPLE — `/ohcreatetimerpost`

```
/ohcreatetimerpost
```

Run with no options, in whichever channel you want the tracker to appear in. Posts the embed there and saves that channel/message so the minute-by- minute updates land on it.

EXAMPLE — `/ohsethallocation`

```
/ohsethallocation location:NE of Wolfshire
```

The "Hal's Moving House" field in the tracker embed now reads "NE of Wolfshire" until updated again.

Setup (running it locally, e.g. on Windows)

1. **Install Node.js** (v18+) from nodejs.org if you don't have it. Then open Command Prompt and confirm it's on your PATH:

   ```
   node -v
   npm -v
   ```

   Both should print version numbers. If you get "not recognized as an internal or external command," reinstall from nodejs.org — the official installer sets up the PATH for you automatically.

   Note: Node.js installs to its own program folder (e.g. `D:\Nodes`) — you never need to open or work inside that folder. Your bot's code (this project) lives in a completely separate folder of your choosing, e.g. `C:\Users\YourName\Desktop\OHbot`.

2. **Create a Discord application & bot**
   - Go to https://discord.com/developers/applications → click your app (or create one with "New Application")
   - **Bot** tab → Add Bot → copy the token → this is `BOT_TOKEN`
   - **General Information** tab → copy the Application ID → this is `CLIENT_ID`
   - **OAuth2** tab → scroll to "OAuth2 URL Generator":
     - Check the **bot** scope (adds it as a bot user) and **applications.commands** scope (required for slash commands)
     - A "Bot Permissions" section appears below — check **Send Messages** and **Embed Links**
     - Copy the URL that gets generated at the bottom of the page, paste it into your browser, and choose your server to invite the bot (you need "Manage Server" permission on that server yourself)

3. **Get your server's GUILD_ID** (optional but recommended — makes slash commands appear instantly instead of waiting up to an hour for global registration)
   - In Discord: User Settings (gear icon) → Advanced → turn on **Developer Mode**
   - Right-click your server's icon in the left sidebar → **Copy Server ID** — that number is your `GUILD_ID`

4. **Configure**
   - In your project folder, open Command Prompt and `cd` into it, e.g.:

     ```
     cd C:\Users\YourName\Desktop\OHbot
     ```

   - Copy the example config to a real one. On Windows Command Prompt, use `copy` (not the Mac/Linux `cp`):

     ```
     copy config.example.json config.json
     ```

   - Open `config.json` in Notepad (`notepad config.json`) and fill in `BOT_TOKEN`, `CLIENT_ID`, `GUILD_ID`, and set `TIMEZONE` to an IANA name (e.g. `America/Chicago`, `Australia/Melbourne`). Leave `CHANNEL_ID` / `MESSAGE_ID` blank — `/ohcreatetimerpost` fills those in.

5. **Install dependencies and run**, both typed into the same Command Prompt window, standing in your project folder:

   ```
   npm install
   node bot.js
   ```

   `npm install` downloads the libraries this project needs (discord.js, cron, moment-timezone) into a `node_modules` folder — it doesn't run anything itself. `node bot.js` is the command that actually starts the bot. If it works, you'll see `Logged in as YourBot#1234` printed and the terminal will stay open and running — that's normal, it needs to stay open for the bot to stay online.

6. In Discord, run `/ohcreatetimerpost` in the channel you want the tracker posted to.

Important: running it this way means the bot is only online while your computer is on, that Command Prompt window stays open, and your internet connection is up. Closing the terminal, sleeping, or shutting down your PC takes the bot offline. See "Hosting it 24/7 in the cloud"

below if you want it to stay up without your computer running.

Hosting it 24/7 in the cloud

To keep the bot online without leaving your own PC running, deploy it to a host like Railway, which has a free monthly usage credit that comfortably covers a bot this size.

1. **Push your code to GitHub** — create a repo and upload the project. Do **not** upload `config.json` (your `.gitignore` already excludes it if you're using git normally) — only `config.example.json`, the `.js` files, `package.json`, and this README should go up.
2. **Create a Railway project** — sign up at railway.com with GitHub, click "New Project" → "Deploy from GitHub repo" → pick your repo. Railway detects it's a Node.js project from `package.json` automatically.
3. **Add your config as one environment variable** — in the service's "Variables" tab, add a variable named `CONFIG_JSON` and paste your entire `config.json` content (with real values filled in) as its value. This bot's `configHandler.js` automatically reads from `CONFIG_JSON` when there's no `config.json` file on disk, so you never need to commit your token to GitHub.
4. **Deploy** — Railway runs `npm install` then `npm start` (`node bot.js`) automatically. Check the "Deployments" tab for build logs, then the runtime logs for `Logged in as YourBot#1234`.
5. **One limitation to know**: Railway's filesystem resets on every redeploy, so anything the bot writes to `config.json` at runtime (like `MESSAGE_ID` after `/ohcreatetimerpost`, or `HAL_LOCATION` after `/ohsethallocation`) is lost on redeploy — just re-run the relevant command afterward. Railway also supports persistent volumes if this becomes annoying.
6. Railway's free tier gives $5/month in usage credit; without a linked card, your project pauses once that's used up each month and resumes next month. A bot this size typically stays well within that credit.

Notes

- `config.json` holds your bot token — never commit or share it (already covered by `.gitignore`).
- Vendor/Commission weekly reset day/time isn't exposed as a slash command (it wasn't asked to be live-adjustable) — edit `COMBINED_RESET_DAY` (0=Sunday…6=Saturday), `COMBINED_RESET_HOUR`, and `COMBINED_RESET_MINUTE` directly in `config.json` and restart the bot if you need to change it. Happy to add a slash command for this too if you'd rather not touch the file.
- The embed only updates the persistent countdown message — it does not post separate "this is happening now" alert messages, per what was asked.